



# 银行账户管理系统设计汇报

答辩小组



小组成员：



时间：





北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY

# 目录

CONTENTS

---

1

基本情况

2

设计与实现

3

测试与结论

4

总结与思考



北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY

# 1

## 基本情况

## 题目内容

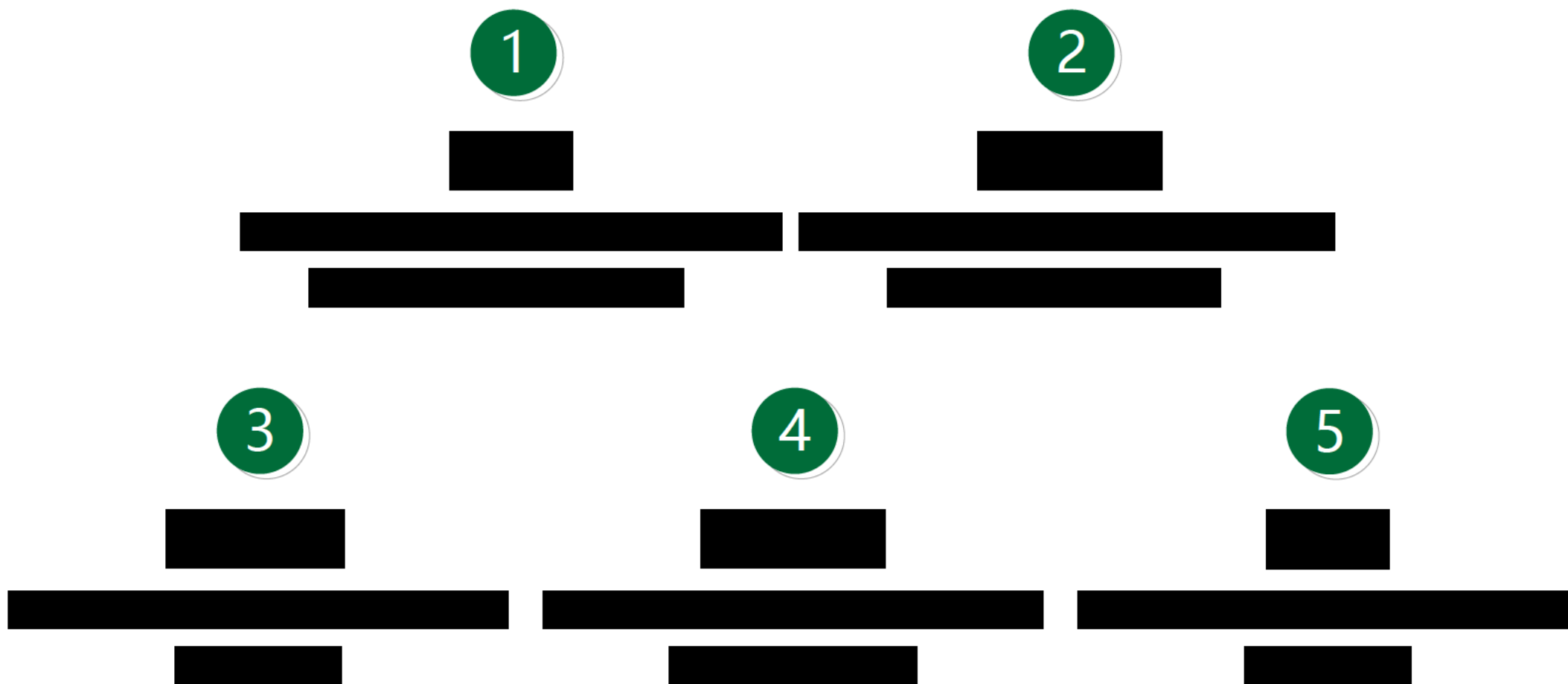
设计并实现一个银行账户管理系统，支持客户和管理员两种身份登录，客户具备开户、查询、修改、存款、取款、转账、销户等功能，管理员还可浏览、查找账户信息等。



## 题目要求及约束条件

银行客户信息以一个账户一条记录的形式存储，每个账户记录包含的信息有身份标识号（ID）、姓名、电话、电子邮箱、身份证号、银行卡号、账户余额等信息。并实现题目要求所有功能。







北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY

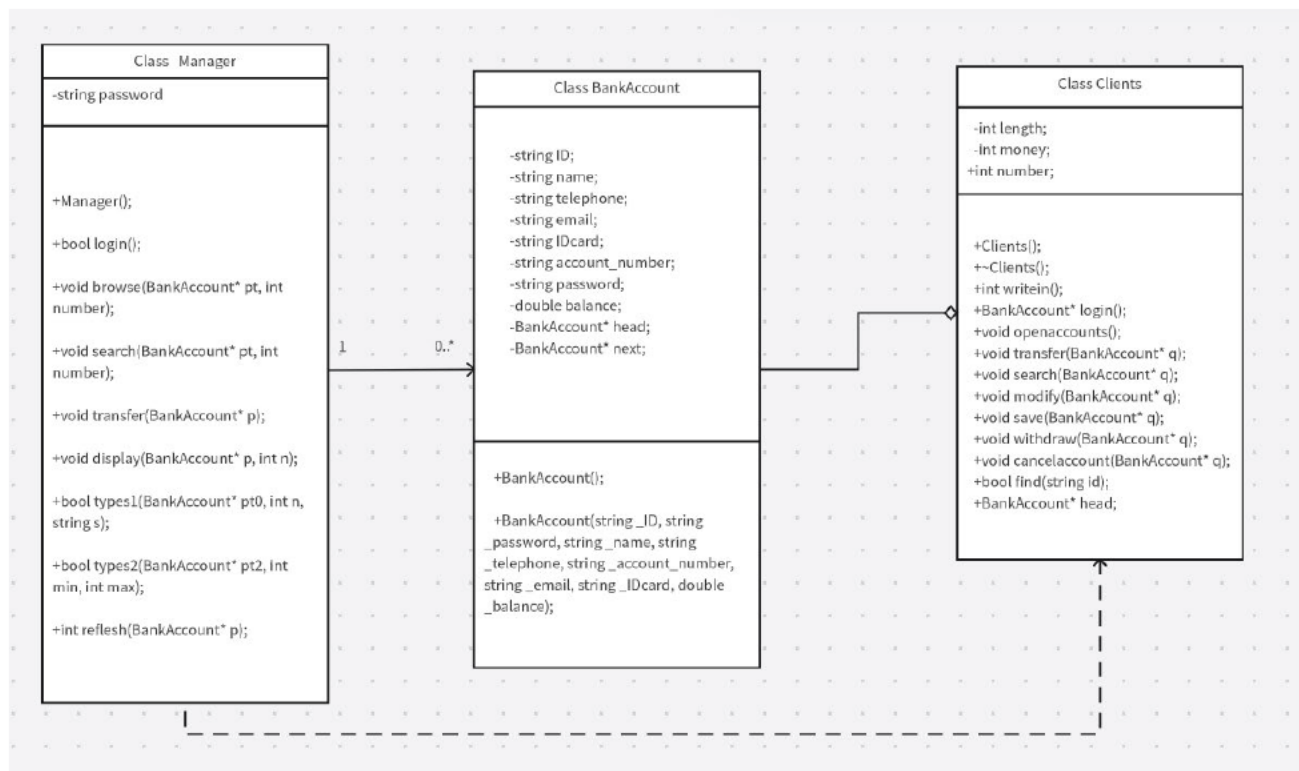
## 2

## 设计与实现

1. 账户类：相当于链表的节点，用于存储账户的基本信息，同时存储指针指向链表的下一个节点。

2. 客户类：定义客户对银行账户的各种操作，并且存放账户数量，存款总额等重要信息

3. 管理员类：定义管理员对银行账户的各种操作。



本系统采用**单向链表**存储账户数据，主要基于以下考虑：

## 动态管理

银行账户数量可能频繁变化，链表支持高效的动态增删操作。使用链表操作可以存储更多的银行账户，不会像数组存在账户数量限制。

## 内存利用率高

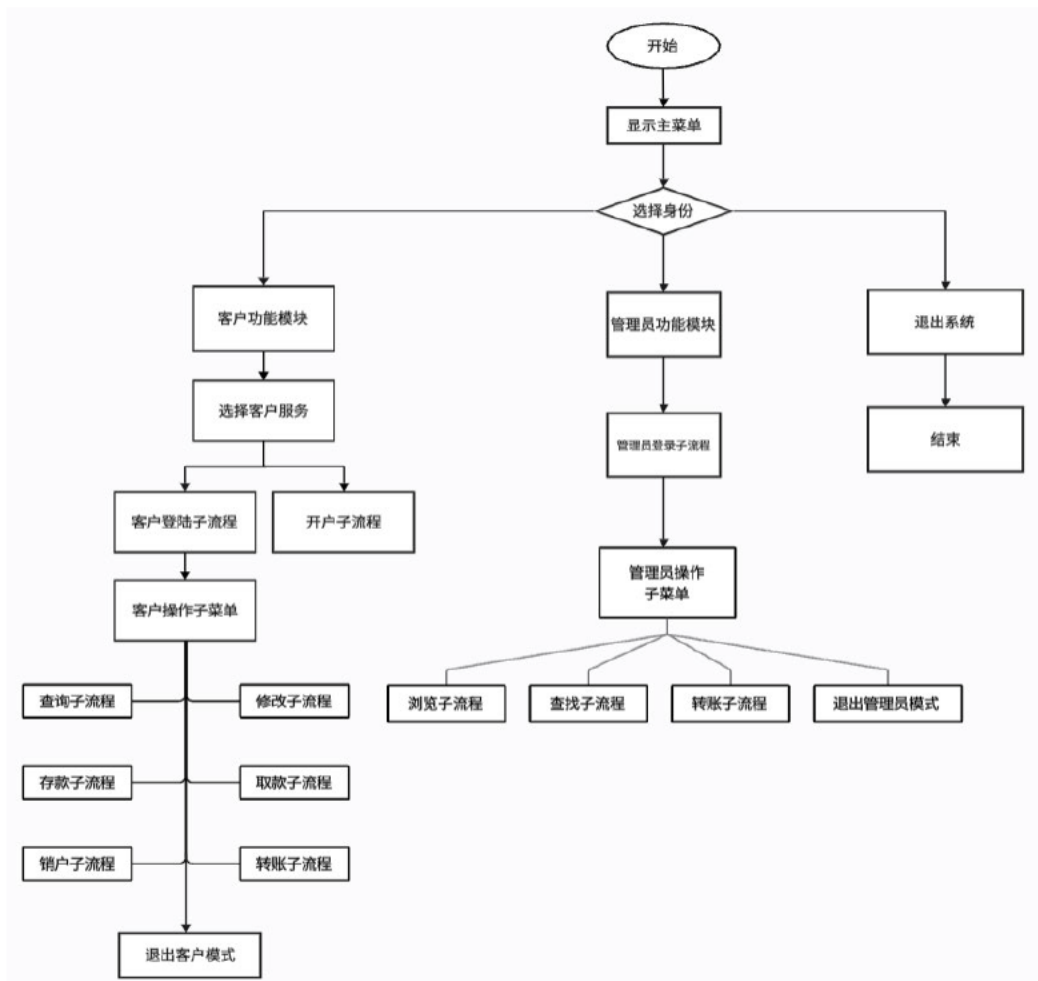
链表无需提前分配内存，避免数组预分配可能造成的空间浪费。

## 实现简单

链表结构易于实现和维护，便于对各种账户信息进行增删改查。

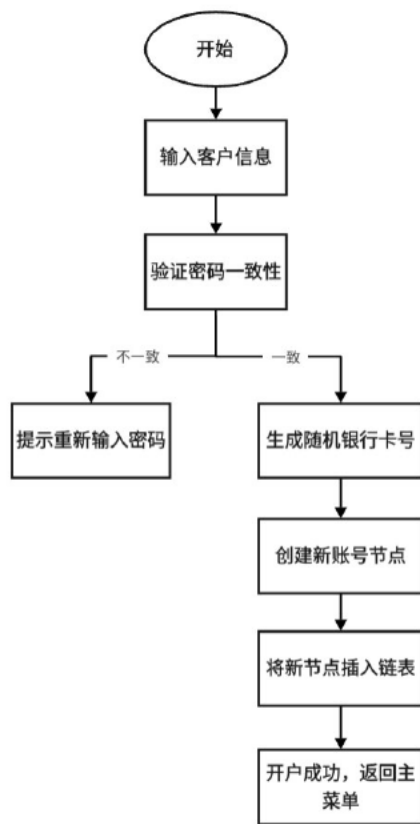


主代码实现银行账户管理系统的主要操作流程，定义了系统的显示界面以及操作方式，并在每一步操作都给予用户充分的提示信息便于用户操作。



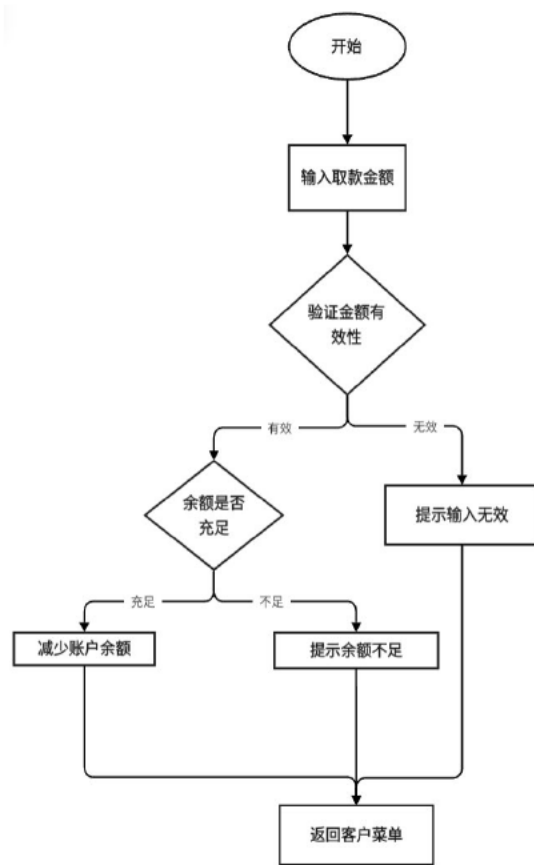
主程序

开户时先输入ID等账户信息，并检验ID的唯一性，与此同时在设置密码时要求用户两次输入密码并验证一致性，增强系统安全性。然后将客户信息作为链表节点插入到表头节点之后完成开户操作。



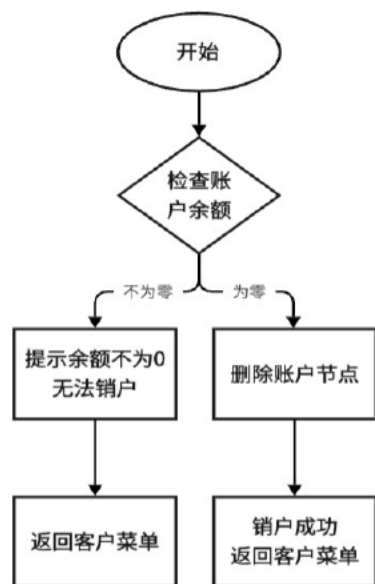
开户子程序

取款时先验证输入金额的有效性，然后检测账户余额是否充足，充足则减少账户余额，反之则提示余额不足。



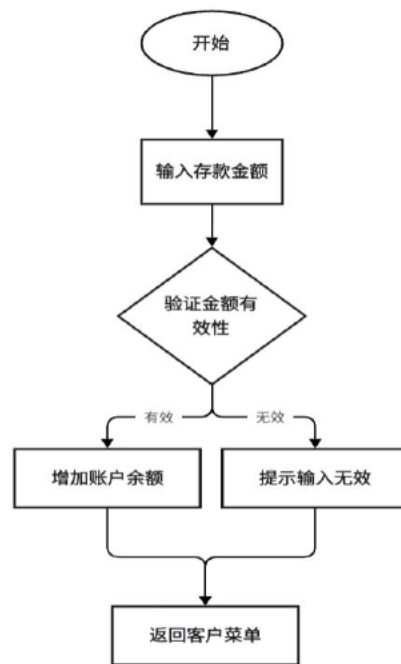
取款子程序

选择销户后，系统先检查账户内是否还有余额，没有则销户，反之提示用户销户失败。



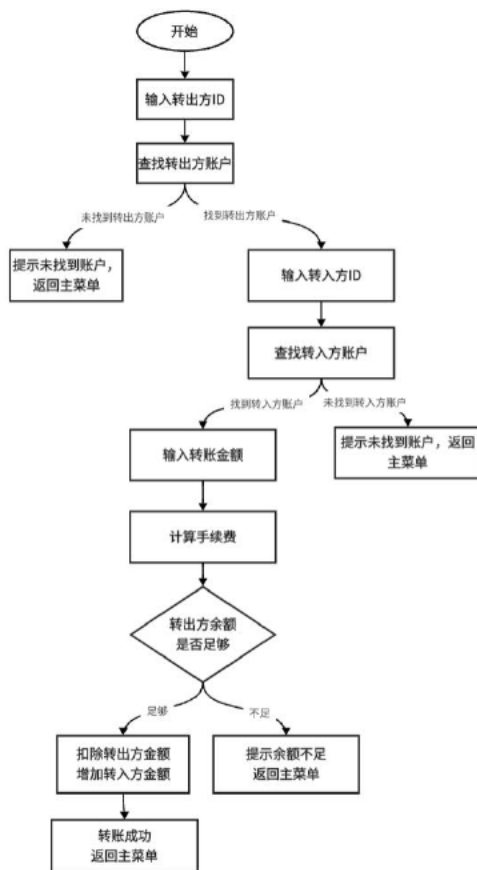
销户子程序

登陆成功后，可选择存款服务，输入金额后验证有效性，成功后则增加账户余额，否则提示用户重新输入。



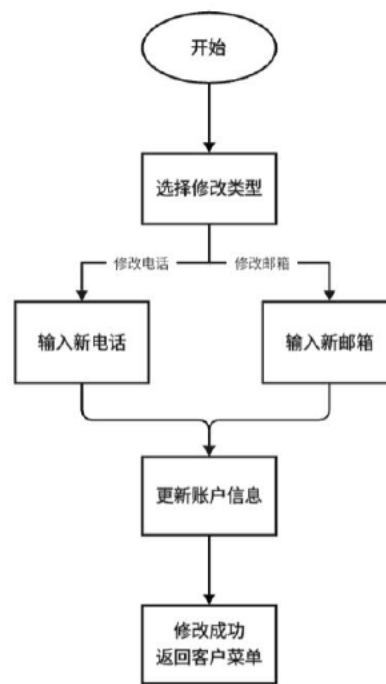
存款子程序

客户与管理员均可进行转账操作。其中客户需要先输入对方银行卡号，若不存在则提示用户不存在。接下来输入转账金额并扣除手续费，双方的存款金额均发生相应变化。管理员转账则需输入转入方和转出方的银行卡号，其余操作与客户一致。



转账子程序

修改仅能修改账户的电话和邮箱，修改之后系统会自动更新账户的信息并保存。



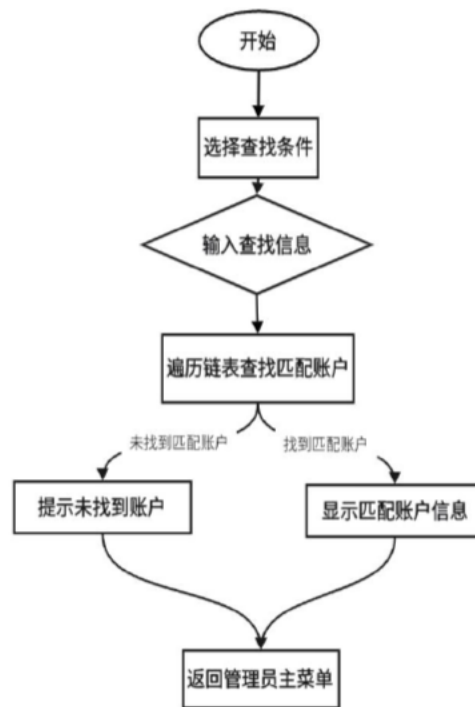
修改子程序

通过遍历链表进行账户信息输出，同时统计客户数量和存款总数。



浏览子程序

选择查找条件，然后遍历链表查找账户，成功找到就显现账户信息，反之就提示未找到账户。



查找子程序

开户功能（链表插入）：将输入的客户信息作为一个节点插入链表的表头节点后，对用户的每一步操作进行提示，同时纠正用户的错误操作。

```
#include "Clients.h"
```

```
#include Clients.h
#include <random>
//开户
void Clients::openaccounts()
{
    system("cls");
    string name0;
    string password01;
    string password02;
    string id0;
    string email0;
    string telephone0;
    string idcard0;
    string account_number = "";
    cout << "请输入您的姓名: " << endl;
    cin >> name0;
    for (;;) {
        cout << "请输入您的密码: " << endl;
        cin >> password01;
        cout << "请再次输入您的密码: " << endl;
        cin >> password02;
        if (password01 != password02) {
            cout << "两次输入密码不一致, 请重新输入: " << endl;
        }
        else break;
    }
    cout << "请输入您的ID号码: " << endl;
    cin >> id0;
    while (find(id0)) {
        cout << "该ID号已被占用, 请重新输入您的ID" << endl;
        id0.clear();
        cin >> id0;
    }
    cout << "请输入您的电子邮箱: " << endl;
    cin >> email0;
    cout << "请输入您的电话号码: " << endl;
    cin >> telephone0;
    cout << "请输入您的身份证号: " << endl;
    cin >> idcard0;
    random_device rd; // 随机设备产生种子
    mt19937 gen(rd()); // 梅森旋转引擎
    // 2. 均匀分布的整数
    uniform_int_distribution<> dist_int(0, 9);
    for (int i = 0; i < 16; i++) {
        account_number += '0' + dist_int(gen);
    }
    cout << "您的银行卡号是: " << account_number << endl;
    /*创建新账户的结点*/
    BankAccount* p = new BankAccount(id0, password01, name0, telephone0, account_number, email0, idcard0, 0.0);
    p->next = head->next;
    head->next = p;
    length++;
    number++;
    cout << "开户成功!" << endl;
}
```



登录查找（链表查找）：通过遍历链表找到对应账户节点

。

```
#include "Clients.h"
bool Clients::find(string id)
{
    BankAccount* pf;
    pf = head;
    while (pf->ID != id && pf->next != NULL) {
        pf = pf->next;
    }
    if (pf->ID == id)
        return true;
    else
        return false;
}
```

销户（链表删除）：  
将一个账户节点从链表中删除，并对销户成功与否进行提示。

```
#include "Clients.h"
/*销户*/
void Clients::cancelaccount(BankAccount* q) {
    system("cls");
    BankAccount* pt = head;
    if (q->balance != 0)
        cout << "余额不为零，无法办理销户！" << endl;
    /*删除用户q所在结点*/
    else {
        while (pt->next != q && pt != NULL) {
            pt = pt->next;
        }
        if (pt == NULL) {
            cout << "系统错误！" << endl;
        }
        if (pt->next == q) {
            pt->next = q->next;
            delete q;
            cout << "销户成功！" << "\n";
        }
    }
    length--;
    number--;
}
```



北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY

3

## 测试与结论

输入必要信息进行开户，银行卡号随机生成

```
E:\C++demo\bank\x64\Debug >
请输入您的姓名:
111
请输入您的密码:
111
请再次输入您的密码:
111
请输入您的ID号码:
111
该ID号已被占用, 请重新输入您的ID
222
请输入您的电子邮箱:
222
请输入您的电话号码:
222
请输入您的身份证号:
222
您的银行卡号是: 2552773736528579
开户成功!

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
|
```

客户开户

使用卡号和密码登录账户后可以查看该客户所有信息。

```
E:\C++demo\bank\64\Debug x + v
姓名: 111
身份标识号(ID): 222
电话号码: 222
银行卡号: 2552773736528579
电子邮箱: 222
身份证号: 222
账户余额: 0

-----

请选择您接下来的操作:
1. 继续办理其他业务
2. 退出
```

## 查询

登录账户后可以选择修改电话和电子邮箱，直接输入新电话或电子邮箱就可修改成功。

```
E:\C++demo\bank\64\Debug x + v
请选择您要修改的信息:
1. 电话 2. 电子邮箱
1
请输入新的电话号码:
222
修改成功!

-----

请选择您接下来的操作:
1. 继续办理其他业务
2. 退出
```

## 修改

登录账户，输入存款数字即可存款

```
E:\C++demo\bank\x64\Debug x + v
请输入存款金额:
1000
成功存入人民币1000元!

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
```

存款

登录账户，此时账户中必须有一定数量存款方可取出，否则无效；取款失败可以选择重新输入数额或退出

```
E:\C++demo\bank\x64\Debug x + v
请输入取款金额:
100
成功取出人民币100元!

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
```

取款成功

```
E:\C++demo\bank\x64\Debug x + v
请输入取款金额:
1000
余额不足,请选择接下来的操作:
1.重新输入
2.退出
```

取款失败

若是账户中有余额，则无法销户

```
E:\C++demo\bank\64\Debug x + v - □ ×
余额不为零，无法办理销户！
-----
请选择您接下来的操作：
1.继续办理其他业务
2.退出
```

销户失败

无余额则销户成功

```
E:\C++demo\bank\64\Debug x + v - □ ×
销户成功！
-----
请选择您接下来的操作：
1.继续办理其他业务
2.退出
```

销户成功

没有账户时输出

```
E:\C++demo\bank\64\Debug x + v - □ ×
账户数量：0
总存款数：0
系统中不存在账户！
-----
请选择您接下来的操作：
1.继续办理其他业务
2.退出
```

无账户浏览

有账户时输出，可见账户数量、总存款数、所有账户信息

```
E:\C++demo\bank\64\Debug x + v - □ ×
账户数量：1
总存款数：899.9
序号 ID 姓名 电话 电子邮箱 身份证号 银行卡号 账户余额
1 111 111 222 111 111 3632665644120670 899.9
-----
请选择您接下来的操作：
1.继续办理其他业务
2.退出
```

有账户浏览



这是通过ID遍历  
链表进行查找，  
还能通过姓名，  
电话等进行查找  
，输出账户所有  
信息

```
E:\C++demo\bank\x64\Debug x + v
请输入要查找的ID:
111
序号 ID      姓名      电话      电子邮箱      身份证号      银行卡号      账户余额
1    111      111      222      111      111      3632665644220670  899.9

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
```

查找

```
E:\C++demo\bank\x64\Debug x + v
账户数量: 2
总存款数: 899.9
序号 ID      姓名      电话      电子邮箱      身份证号      银行卡号      账户余额
1    222      222      222      222      4343802549696564  0
2    111      111      222      111      3632665644220670  899.9

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
```

转账前账户

```
E:\C++demo\bank\x64\Debug x + v
请输入转入方的银行卡号:
4343802549696564
请输入要转账的金额:
800
转账成功!
本次转账的手续费为: 0.8

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
```

转账操作

```
E:\C++demo\bank\x64\Debug x + v
账户数量: 2
总存款数: 899.1
序号 ID      姓名      电话      电子邮箱      身份证号      银行卡号      账户余额
1    222      222      222      222      4343802549696564  800
2    111      111      222      111      3632665644220670  99.1

-----

请选择您接下来的操作:
1.继续办理其他业务
2.退出
```

转账后账户



北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY

## 4 总结与思考

## 题目难点要点

1. 链表动态管理账户的增删查改，需要严谨的指针管理，稍有不慎会导致内存泄漏或指针指向错误等问题从而导致系统崩溃

2. 设计系统时要充分考虑用户的使用体验感，为用户的每一步操作提供充分的提示信息，并且可以防止由于误操作导致的错误

3. 文件读写必须与内存数据实时同步，否则可能导致数据丢失

4. 客户和管理员的操作权限需严格区分（如管理员可浏览所有账户）

## 本组工作特点

本组设计了一个银行账户管理系统，实现了开户、查询、存款、取款、转账、销户等核心功能，并通过对文件的读写操作提供了完整的数据备份与恢复功能。本组的工作主要有以下几个特点：

模块化增强

将客户(Clients)和管理员(Manager)功能分离，通过不同类实现，代码结构更清晰。使用 `client.writein()` 进行银行账户的文件读入以初始化数据，避免主函数臃肿。

用户交互优化

我们设计的银行账户管理系统对用户的每一步操作都有充分的提示信息，并且对用户的误操作进行及时的纠正，从而提升了用户的体验感。与此同时，增加输入校验（如 `while (identity < 1 || identity > 3)`），防止无效输入导致程序崩溃。通过 `system("cls")` 清屏以提升界面整洁度。

数据持久化

在退出时调用 `manager.refresh(client.head)` 保存数据到文件，避免数据丢失，提升管理系统的安全性。客户登录失败后能安全退出（`if (p == NULL) break`），避免空指针异常。

这些特点使得输入校验和错误处理更加完善，模块化设计增强了程序的可拓展性，便于后续新增功能，连贯的业务流程更加接近真实网银系统。

## 本组改进方向

1. 程序编写时可能未考虑到随机生成的16位卡号是否会出现相同的情况。

2. 账户安全性有待增强：输入的密码为明文显示，且没有登录失败的次数限制。改进思路可以将密码输入隐藏显示（如\*\*\*\*），连续输入错误密码后锁定账户

3. 编写的管理系统的各种操作仍通过命令行窗口实现，可考虑使用图形界面进一步简化用户的操作。



# 感谢观看